

repositories where alternative AI coding tools may have been used based on repository files (e.g., those with `.vscode` folder may have used GitHub Copilot). We find 382 repositories from the treatment group that may use other AI tools during our observation period: 345 with GitHub Copilot, 63 with Claude Code, 37 with Windsurf, 13 with Cline (13), and 2 with OpenHands (note that they overlap heavily). We build a *Cursor and Other* subset for these repositories and an *Only Cursor* subset for the remaining repositories, along with their own matched controls. The results (Figure 4, Row 3) show that while contamination weakens our main ITT estimates (e.g., some repositories may be using GitHub Copilot before Cursor), all of our main findings are persistent through all three settings and amplified in settings where prior AI tool usage is less likely.

To summarize, the above robustness checks reassure our main causal findings against concerns from potential non-compliance (i.e., treatment repositories not actually using Cursor), selection bias (i.e., treatment repositories generally becoming inactive fast), and confounding from other AI tools (i.e., the presence of other AI tools nullifying estimates on the adoption of Cursor).

5 Discussion

5.1 Theoretical Implications

Our study contributes to the rapidly growing literature regarding the impact of AI assistance on developer productivity [27, 40, 46, 63, 66, 73, 91, 94, 102, 105, 108, 111, 113]. More importantly, our study provides a novel longitudinal lens into project-level macro-outcomes, effectively connecting our findings to the existing software engineering literature around development velocity and software quality [28, 38, 72, 85, 104]. In this section, we connect our findings with prior research and discuss our theory around how LLM agent assistants may impact software development (Figure 1).

5.1.1 The Transient Velocity Gains and Possible Causes Behind It. Our first longitudinal finding—that the project-level velocity gains from adopting Cursor are concentrated in the initial one or two months before returning to a baseline level—contrasts task-level productivity improvements reported in controlled experiments [e.g., 92, 94]. One reason for such contrast likely stems from the temporal dynamics between development velocity and software quality (which is only observable in a longitudinal study setting): While LLM agent assistants increase development velocity, the increase in development velocity itself may increase codebase size and cause accumulation of technical debt; the latter would consequently decrease development velocity in the future. This negative effect of technical debt is supported by both the prior literature [28, 38] and our panel GMM models (Table 3). However, this mechanism alone likely does not fully explain why the development velocity gain vanishes after two months: A $\sim 3\times$ increase in code complexity or a $\sim 5\times$ increase in static analysis warnings would be necessary to fully cancel out the effect of Cursor adoption according to our models (Table 3), which is unlikely.

Another highly plausible explanation, as indicated in Section 4.3, is that open-source developers experience an *excitement-frustration-abandonment cycle* while they adopt Cursor. For example, during the initial adoption phase, developers may experience novelty effects and actively experiment on tasks where AI excels (e.g., rapid

prototyping), contributing to the immediate velocity spike post-adoption. However, as developers encounter scenarios where AI is still limited (e.g., debugging intricate logic, understanding existing codebases, handling edge cases), frustration may accumulate. This frustration, combined with the cognitive overhead of verifying and debugging AI-generated suggestions, could lead to reduced usage or complete abandonment. This interpretation aligns with the emerging qualitative research documenting developer challenges with AI-assisted coding [27, 37, 76] and anecdotal evidence from Cursor users [e.g., 9, 11].

5.1.2 The Accumulation of Technical Debt and Code Complexity. Our findings reveal a nuanced relationship between velocity and quality that challenges simplistic narratives about AI coding degrading code quality [6]. While the absolute levels of static analysis warnings increase post adoption (Finding 2), a large part of this observed effect can be attributed to the causal path of increased velocity $\rightarrow$ increased code base size $\rightarrow$ increased technical debt (Table 3). In other words, LLM agent assistants amplify existing velocity-quality dynamics by enabling faster code production, but may not necessarily introduce more code quality issues than non-adopting projects moving with the same velocity. This proportional relationship has an important practical implication that quality assurance needs to scale with AI-era velocity (see Section 5.2). After all, the use of AI does not change the fact that all code is a liability and the asset lies only in the code’s capabilities [13].

The substantial average increase in code complexity (25.1%, Table 2) warrants particular attention, as code complexity represents a distinct quality dimension from code quality issues. That code complexity increases even after accounting for velocity dynamics (Table 3) gives strong evidence that code generated with Cursor in our study sample is inherently more complex than human-written code. This effectively creates a “complexity debt” in projects that use AI heavily, which may amplify frustration and maintenance costs when the AI fails on more complex codebases later, possibly providing another mechanism explaining the transient velocity gain we observe after Cursor adoption in our dataset.

While the adoption of Cursor leads to no significant changes in duplicate line density in the entire study sample (Table 2, Figure 3), heavy Cursor adopters may exhibit modest increases (Figure 4). Future research is needed to gather evidence on code duplication concerns in high AI usage scenarios.

5.1.3 Contextual Factors in Open-Source Settings. Our findings should be interpreted within the specific context of open-source software development, which differs from enterprise settings in ways that likely influence the patterns we observe. Open-source projects typically feature: (1) voluntary participation with low switching costs, enabling easy abandonment when tools prove frustrating; (2) distributed collaboration with varying levels of coordination, potentially reducing systematic code review that might catch AI-introduced defects; (3) intrinsic motivation and learning goals, where experimenting with AI tools provides value beyond pure productivity; and (4) resource constraints that may limit comprehensive testing and quality assurance regardless of development velocity. These contextual factors likely amplify the excitement-frustration-abandonment cycle while potentially dampening the